CRUD, HTTP Verbs, Status Codes & Idempotency

11 min read Updated 24 Jul 2026 Topic 40 of 70

Bookmark Mark done

Why it matters

The difference between an API that feels professional and one that feels amateur is almost entirely in the details of this topic: choosing the right HTTP method, returning the right status code, and understanding idempotency. These are not pedantic niceties. A wrong status code silently breaks clients: a browser will happily *cache* a response you returned as 200 when it should have been an error, a monitoring dashboard counting "5xx" will miss your outage because you returned 200 with `{"error": true}`, and a payment client that retries a non-idempotent request can charge a customer twice.

Interviewers lean on this topic heavily because it is a fast, reliable filter. "What status code do you return when a user tries to create a resource that already exists?" separates people who have shipped real APIs from people who have only followed tutorials. "Is PUT idempotent? Is POST? Why?" is one of the most common backend questions in existence, and getting it right — with the *reasoning*, not just the answer — signals seniority. Idempotency in particular is the backbone of reliable distributed systems: networks fail, clients retry, and only idempotent operations survive retries without corruption.

Master this and you will write APIs that integrate cleanly with HTTP's built-in machinery — caching, conditional requests, retries, load balancers — instead of fighting it.

The concept, simply

CRUD — Create, Read, Update, Delete — is the set of four operations most resources support. HTTP gives you methods that map naturally onto them, and each operation has a conventional *success* status code.

CRUD to HTTP verbs and status codes

CRUD	HTTP verb	Success code	Idempotent?
Read	GET	200 OK	yes · safe
Create	POST	201 Created	no
Update	PUT / PATCH	200 / 204	PUT yes · PATCH no
Delete	DELETE	200 / 204	yes

Safe = no state change; idempotent = same effect if repeated N times

Each CRUD operation maps to an HTTP verb with a conventional success code. Note which verbs are safe and which are idempotent.

- **Create** → `POST /users` → **201 Created** (plus a `Location` header pointing at the new resource).
- **Read** → `GET /users/42` → **200 OK** (or 404 if it doesn't exist).
- **Update** → `PUT /users/42` (full replace) or `PATCH /users/42` (partial) → **200 OK** with the body, or **204 No Content** if you return nothing.
- **Delete** → `DELETE /users/42` → **204 No Content** (or 200 with a body).

Two properties classify these verbs. A method is **safe** if it does not change server state — GET is safe. A method is **idempotent** if making the same call once or many times leaves the server in the same final state. GET, PUT, and DELETE are idempotent; POST is not. That single distinction drives an enormous amount of correct API design.

How it works (deep)

Idempotency, precisely. Idempotent means *the effect on server state* of N identical requests equals the effect of one. It is *not* about the response being byte-identical. Consider `DELETE /users/42`: the first call deletes the user and returns 204; the second call finds nothing to delete and returns 404. The *responses differ*, but the *server state is identical* after either the first or the second call — user 42 is gone. That still counts as idempotent. This nuance trips up almost everyone; interviewers specifically probe it.

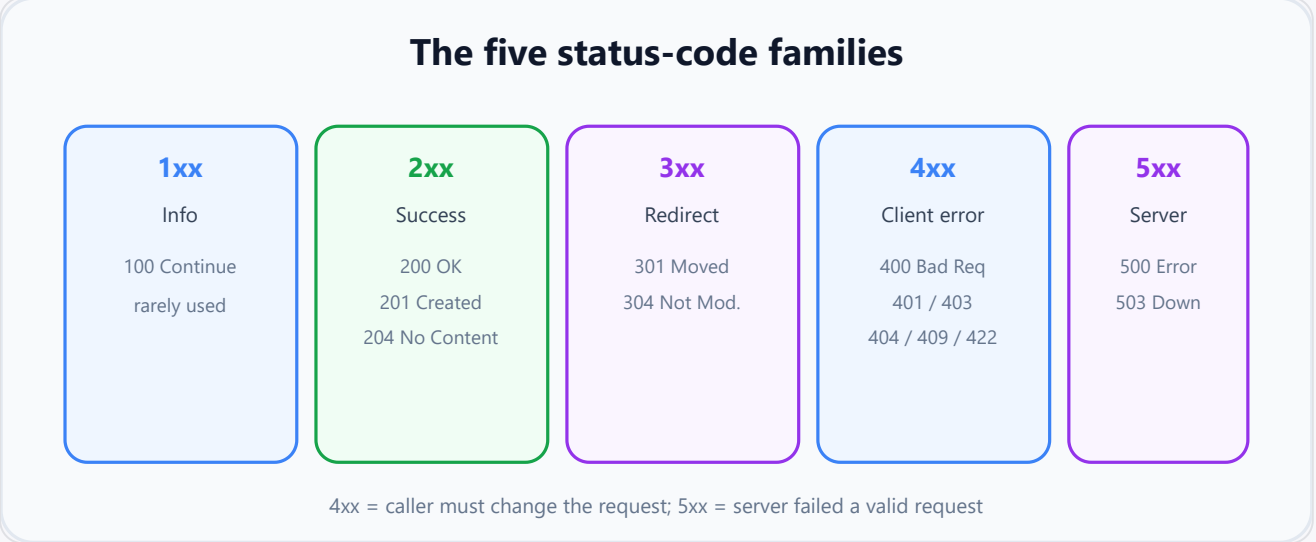
call changes state each time. That is exactly why forms warn "do not click submit twice" and why retrying a failed POST is dangerous — you cannot know whether the first one actually succeeded before the network dropped.

Why PUT is idempotent but POST is not, even though both "write." PUT means "make the resource at this URL equal to this representation." `PUT /users/42` with a body sets user 42 to that exact state; do it ten times and user 42 still holds that same state. The client chooses the URL/identity, so repetition converges. POST lets the *server* choose the identity, so repetition diverges. This is the crux.

PATCH is usually not idempotent. `PATCH` applies a partial change. `{ "op": "increment", "field": "views" }` is clearly not idempotent — each call adds one. Even a "set field X to Y" PATCH is only idempotent by convention, and JSON Patch operations like "add to array" are not. Treat PATCH as non-idempotent unless you have specifically designed it to be.

Idempotency keys. Since POST is not idempotent but networks retry, robust APIs (Stripe is the canonical example) accept an `Idempotency-Key` header. The client generates a unique key per logical operation; the server stores "I already processed key K and here is the result." A retry with the same key returns the stored result instead of creating a second order or charge. This turns an unsafe POST into a safely-retriable one — a favorite senior-level design question.

Choosing the right status code matters as much as the verb.



The families: **2xx** success, **3xx** redirection/caching, **4xx** client error, **5xx** server error. The single most important line is between 4xx and 5xx: *4xx is the caller's fault* (bad input, missing auth), *5xx is your fault* (an exception, a downstream failure). Getting this wrong wrecks monitoring and on-call. The high-value 4xx codes to know cold: **400** malformed request, **401** not authenticated, **403** authenticated but not allowed, **404** not found, **409** conflict (e.g. duplicate/edit collision), **422** well-formed but semantically invalid (validation failed), **429** too many requests. On the success side: **200** OK, **201** Created, **204** No Content, **304** Not Modified.

401 vs 403 is a classic trap. 401 = "I don't know who you are" (no or bad credentials) — log in. 403 = "I know who you are and you still can't do this" — logging in again won't help. **400 vs 422** is another: 400 for a request that is syntactically broken (unparseable JSON), 422 for one that parses fine but fails business validation (email already taken, age negative). And **409 Conflict** is the right code when the request collides with current state — creating a user whose email already exists, or editing a record someone else changed.

Code examples

A full CRUD controller with correct verbs and status codes. Note `201` plus `Location` on create, and `204` on delete.

JAVASCRIPT Copy

```
const express = require('express');
const router = express.Router();

// CREATE — 201 + Location header, body is the new resource
router.post('/users', async (req, res) => {
  const user = await User.create(req.body);
  res.status(201)
     .location(`/users/${user.id}`)
     .json(user);
});

// READ one — 200 or 404
```

```
  res.status(200).json(user);
});

// REPLACE — PUT is idempotent: set the resource to this exact state
router.put('/users/:id', async (req, res) => {
  const user = await User.replace(req.params.id, req.body);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.status(200).json(user);
});

// DELETE — idempotent; 204 with empty body
router.delete('/users/:id', async (req, res) => {
  await User.remove(req.params.id); // no-op if already gone
  res.status(204).end();
});

module.exports = router;
```

Returning the right 4xx for the right reason — distinguishing 401, 403, 409, and 422:

JAVASCRIPT Copy

```
router.post('/orders', authenticate, async (req, res) => {
  // 403: authenticated but not permitted
  if (!req.user.canOrder) {
    return res.status(403).json({ error: 'Not allowed to place orders' });
  }
  // 422: well-formed but semantically invalid
  if (req.body.quantity ≤ 0) {
    return res.status(422).json({ error: 'Quantity must be positive' });
  }
  // 409: conflicts with current state
  const existing = await Order.findByIdempotencyRef(req.body.ref);
  if (existing) {
    return res.status(409).json({ error: 'Order already exists', id: exist
  }
  const order = await Order.create(req.body);
```

Making an unsafe POST safely retriable with an idempotency key — the pattern Stripe uses:

JAVASCRIPT Copy

```
router.post('/charges', async (req, res) => {
  const key = req.headers['idempotency-key'];
  if (!key) return res.status(400).json({ error: 'Idempotency-Key required

  // Already processed this key? Return the SAME result — no double charge
  const prior = await IdempotencyStore.get(key);
  if (prior) return res.status(prior.status).json(prior.body);

  const charge = await paymentGateway.charge(req.body);
  const result = { status: 201, body: charge };

  await IdempotencyStore.save(key, result); // persist keyed on the client
  res.status(result.status).json(result.body);
});
```

A PATCH that is deliberately *non-idempotent* — incrementing a counter — to make the property concrete:

JAVASCRIPT Copy

```
// Each call adds 1: calling it 3 times ≠ calling it once. NOT idempotent
router.patch('/posts/:id/views', async (req, res) => {
  const post = await Post.increment(req.params.id, 'views', 1);
  res.status(200).json({ views: post.views });
});
```

Common mistakes & gotchas

- **Returning 200 for everything**, including errors, with an `{ "error": ... }` body. This breaks HTTP-aware clients, caches, and monitoring. Use real status codes.
- **Confusing 401 and 403:** 401 = not authenticated (who are you?); 403 = authenticated but forbidden (you can't do this). Logging in fixes 401, never 403.
- **Using 400 for validation failures** when 422 is more precise: 400 = unparseable; 422 = parsed but invalid.
- **Thinking "idempotent" means "same response."** It means same *server state*. DELETE returning 404 on the second call is still idempotent.
- **Retrying POST blindly:** without an idempotency key, a retried create can duplicate orders/charges.
- **Using PUT for partial updates:** PUT replaces the whole resource, so omitted fields get wiped. Use PATCH for partial changes.
- **201 without a `Location` header:** clients can't discover where the new resource lives.
- **Using GET with side effects:** GET must be safe; prefetchers and crawlers will trigger those side effects.

Best practices

- Map CRUD to verbs faithfully: POST create, GET read, PUT/PATCH update, DELETE delete.
- Return **201 + Location** on create, **204** when there's genuinely no body to return, and **200** when you return the resource.
- Keep the **4xx/5xx boundary honest** — it is the contract your monitoring and on-call rely on.
- Prefer **422** for validation errors, **409** for state conflicts, and distinguish **401** from **403** precisely.
- Make writes **retry-safe**: PUT/DELETE are naturally idempotent; wrap POST create/charge flows with an `Idempotency-Key`.
- Keep GET **safe** — never mutate state on a GET.
- Return a consistent, machine-readable error body alongside the status code (covered in the versioning topic).

Q: Is PUT idempotent? Is POST? Why?

PUT is idempotent: it sets the resource at a client-chosen URL to an exact state, so repeating converges. POST is not: it creates a new resource with a server-chosen identity each time, so repeating diverges (two orders). The reasoning — who chooses the identity — is what they want, not just the yes/no.

Q: DELETE returns 204 the first time and 404 the second. Is it still idempotent?

Yes. Idempotency is about final *server state*, not identical responses. After either call, the resource is gone — same state. The differing status codes are fine. This is the classic trap.

Q: 401 vs 403?

401 = not authenticated ("I don't know who you are") — provide/refresh credentials. 403 = authenticated but not authorized ("I know you, you still can't") — re-auth won't help. Returning 403 when you mean 401 misleads clients into not retrying login.

Q: Which status code for "email already registered" on signup?

409 Conflict — the request conflicts with current server state. Some teams use 422 (validation). Either is defensible if consistent; 200 or 400 are not. Explaining the 409-vs-422 nuance scores well.

Q: How do you make a "create payment" endpoint safe to retry?

Accept an `Idempotency-Key` header. Store the result keyed by that key; a retry with the same key returns the stored result instead of charging again. This makes a non-idempotent POST effectively idempotent — the Stripe pattern.

Because HTTP status codes are the machine-readable contract for caches, load balancers, retry logic, and monitoring. A 200 error is invisible to all of them — your dashboards show green during an outage, and clients cache failures. Use the status line; put detail in the body.

Cheat-card

- **Create** POST → 201 + Location · **Read** GET → 200/404 · **Update** PUT/PATCH → 200/204 · **Delete** DELETE → 204.
- **Safe:** GET (no state change). **Idempotent:** GET, PUT, DELETE. **Not idempotent:** POST, (usually) PATCH.
- Idempotent = same *server state* after N calls, not same response.
- **401** = who are you? · **403** = you can't · **404** = not here · **409** = conflict · **422** = invalid · **429** = rate limited.
- **4xx** = caller's fault · **5xx** = server's fault. Keep the line honest.
- Retry-safe POST = `Idempotency-Key` header + server-side result store.
- PUT replaces the whole resource; PATCH changes part of it.

QUICK CHECK — no marks, just for you.

1. DELETE /users/42 returns 204 the first time and 404 the second. Is the endpoint still idempotent?

- ◯ No, because the two calls returned different status codes
- ◯ No, because a 404 means the second call changed state again
- ◯ Only if the server returns 204 both times
- ◯ Yes, because idempotency is about final server state, not identical responses, and the user is gone after either call

2. Why is PUT idempotent while POST is not?

- ◯ PUT sets a client-chosen URL to an exact state so repeats converge, while POST creates a server-assigned identity each time so repeats diverge
- ◯ PUT never writes to the database, so repeating it changes nothing
- ◯ POST is idempotent too; only PATCH is non-idempotent
- ◯ PUT is cached by the browser, which prevents duplicate writes

Check answer

3. What is the difference between HTTP 401 and 403?

- ◯ 401 means the server crashed; 403 means the client sent bad JSON
- ◯ 401 means not authenticated ("I don't know who you are"); 403 means authenticated but not permitted
- ◯ 401 means forbidden forever; 403 means retry after logging in
- ◯ They are interchangeable synonyms for "access denied"

Check answer

Node.js Backend Kit

LEARN MORE

ARTICLES & DOCS

HTTP response status codes
developer.mozilla.org

HTTP request methods
developer.mozilla.org

VIDEOS



Learn HTTP Status Codes In 10 Minutes
Web Dev Simplified

Hand-picked references for this lesson.